KONGU ENGINEERING COLLEGE

(AUTONOMOUS)

PERUNDURAI, ERODE – 638 060

EXPERIMENT - 6

PROJECT TITLE

PREDICTIVE MAINTENANCE AND MACHINE HEALTH MONITORING

SUBMITTED BY

ABISHEK R S - 24MER002

GOWRISANKAR K - 24MER010

NAVEENRAJ K - 24MER035

PRAVEEN N M - 24MER043

PREDICTIVE MAINTENANCE AND MACHINE HEALTH MONITORING

1. Project review:

The Predictive Maintenance and Machine Health Monitoring System is a compact prototype designed to continuously monitor the operating condition of a machine and identify abnormal conditions at an early stage. The system uses an ESP8266 NodeMCU as the central controller along with sensors to monitor important machine parameters such as temperature and vibration.

The sensors continuously measure the machine condition and send the readings to the ESP8266 NodeMCU. The controller processes these values and compares them with predefined normal operating limits. When the measured values exceed the selected limits, the system identifies a possible abnormal machine condition and provides an alert using a buzzer and LED.

The proposed system provides a simple, low-cost, and automatic method for machine condition monitoring. It can help in identifying potential problems before complete machine failure occurs. The system can also be further developed with Wi-Fi connectivity, cloud monitoring, data logging, mobile notifications, and advanced predictive maintenance analysis.

2. Problem Statement:

Machines used in industries and mechanical systems may experience problems such as excessive vibration, overheating, bearing failure, and abnormal operating conditions. If these problems are not identified at an early stage, they can result in unexpected machine failure, production loss, increased maintenance cost, and equipment damage.

Traditional maintenance methods may depend on periodic manual inspection or maintenance after a failure has occurred. These methods may not provide continuous information about the actual condition of the machine.

Therefore, there is a need for a simple and low-cost system that can continuously monitor important machine parameters and provide an early warning when abnormal conditions occur.

The proposed system addresses these problems by integrating sensors, ESP8266 NodeMCU, buzzer, LED, and an optional relay module. The sensors monitor the machine condition, while the ESP8266 processes the readings.

3. Proposed Solution:

The proposed solution consists of an ESP8266-based Predictive Maintenance and Machine Health Monitoring System. The system uses sensors to continuously monitor important machine operating parameters.

A temperature sensor is used to monitor the machine temperature, while a vibration sensor is used to detect abnormal vibration conditions. The sensor readings are sent to the ESP8266 NodeMCU for processing.

The ESP8266 compares the measured values with predefined threshold limits. If the temperature or vibration level exceeds the selected limit, the system activates a buzzer and LED to provide an alert. A relay can also be used to demonstrate automatic machine shutdown or safety control.

Proposed Operation:

1. Start the system and initialize the ESP8266.
2. Initialize the temperature sensor, vibration sensor, buzzer, LED, and relay.
3. The sensors continuously monitor the machine condition.
4. Temperature and vibration data are sent to the ESP8266.
5. ESP8266 processes the sensor readings.
6. The measured values are compared with predefined safety limits.
7. If the machine parameters are normal, the system continues monitoring.
8. If abnormal vibration or high temperature is detected, the buzzer is activated.
9. The LED provides a visual warning indication.
10. The relay can be activated for machine safety or shutdown control.
11. The system continuously repeats the monitoring process.

4. System Architecture:

The Predictive Maintenance and Machine Health Monitoring System consists of five main sections:

Input Section:

The input section consists of:

- **Temperature Sensor** – Measures the operating temperature of the machine.
- **Vibration Sensor** – Detects vibration and abnormal machine movement.
- The sensors send the measured data to the ESP8266 NodeMCU.

Processing Section:

The ESP8266 NodeMCU acts as the central controller.

- Receives temperature and vibration sensor data.
- Processes the sensor readings.
- Compares the values with predefined limits.
- Identifies possible abnormal machine conditions.
- Controls the buzzer, LED, and relay according to the programmed condition.

Actuation Section:

The Relay Module can be used for machine protection.

- The relay receives a control signal from the ESP8266.
- It can be used to control a suitable machine load.
- It can demonstrate automatic shutdown during an abnormal condition.

Indication Section:

The Buzzer and LED provide warning indications.

- Buzzer OFF and LED OFF → Machine condition normal.
- Buzzer ON and LED ON → Abnormal machine condition detected.

Connectivity Section:

The ESP8266 has built-in Wi-Fi capability.

The system can be further developed for:

- IoT-based machine monitoring.
- Cloud data storage.
- Remote machine monitoring.
- Mobile notifications.
- Maintenance history recording.
- Predictive analysis using collected machine data.

5. Circuit Table:

Important Pin Mapping:

ESP8266 NodeMCU Pin		Component
GPIO 2	D4	Vibration Sensor → OUT
A0	A0	Temperature Sensor → Analog Output
GPIO 14	D5	Relay Module → IN
GPIO 12	D6	Buzzer → Positive Terminal
GPIO 13	D7	LED → Anode through resistor
3.3V / 5V	–	Sensor Power
GND	–	Common Ground

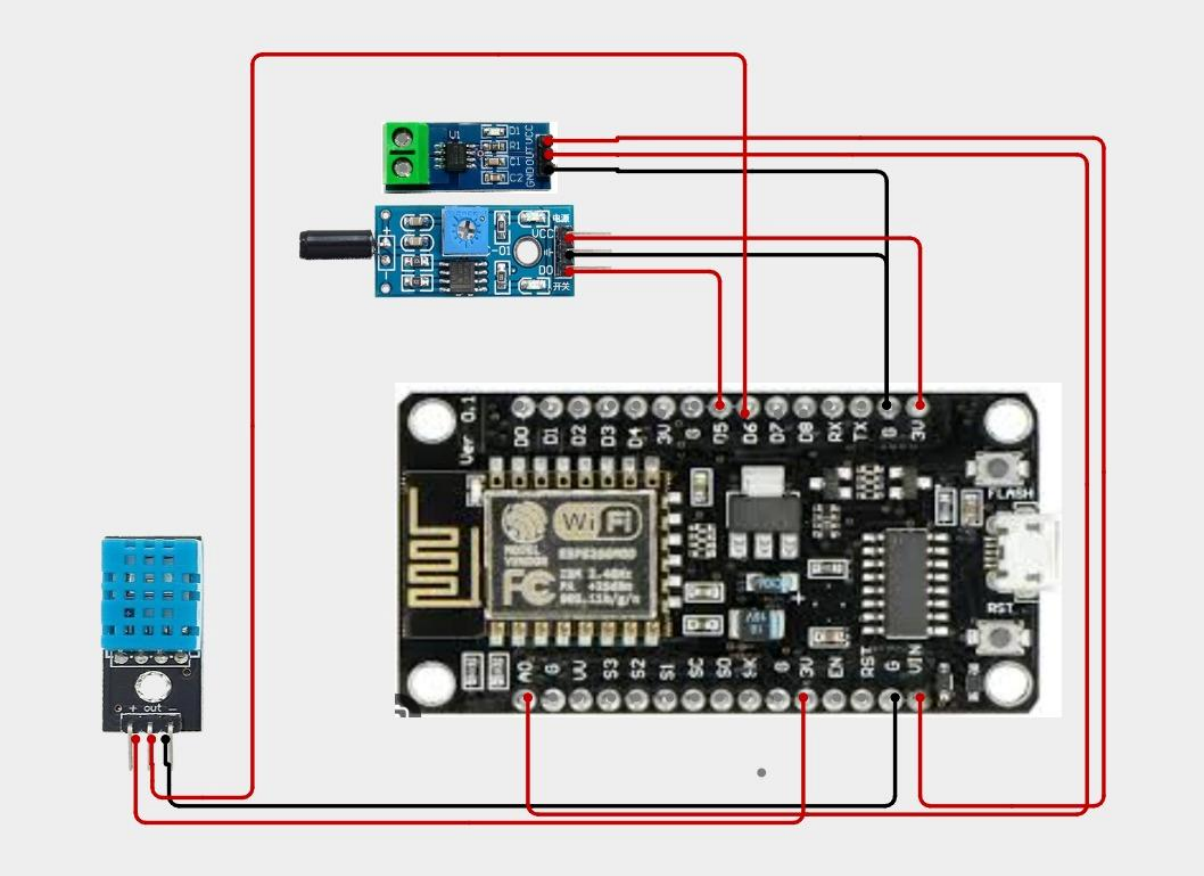
6. Circuit Design:

The circuit consists of an ESP8266 NodeMCU, temperature sensor, vibration sensor, buzzer, LED, relay module, breadboard, and jumper wires.

The temperature sensor is connected to the analog input of the ESP8266 to monitor the machine temperature. The vibration sensor output is connected to GPIO2 (D4) to detect abnormal vibration.

The buzzer is connected to GPIO12 (D6) and provides an audible warning. The LED is connected to GPIO13 (D7) through a suitable resistor and provides a visual indication. The relay module is connected to GPIO14 (D5) and can be used to demonstrate automatic machine shutdown or safety control.

The ESP8266 continuously monitors the sensor readings. When excessive temperature or abnormal vibration is detected, the ESP8266 activates the buzzer and LED. The relay can also be activated to protect the machine from further damage.



7. Algorithm:

1. Start the system.
2. Initialize the ESP8266, temperature sensor, vibration sensor, buzzer, LED, and relay.

3. Read the temperature sensor value.
4. Read the vibration sensor value.
5. Compare the temperature with the predefined limit.
6. Check whether abnormal vibration is detected.
7. If an abnormal condition is detected, turn ON the buzzer, LED, and relay.
8. Display "MACHINE ALERT – MAINTENANCE REQUIRED."
9. Otherwise, keep the buzzer and LED OFF and display "MACHINE HEALTH NORMAL."
10. Wait for a short interval and repeat the monitoring process continuously.

8. Coding:

```
// PREDICTIVE MAINTENANCE AND MACHINE HEALTH MONITORING
// ESP8266 NodeMCU
// Sensors:
// SW-420 Vibration Sensor
// DHT11 Temperature/Humidity Sensor
// ACS712 Current Sensor
// + WiFi + ThingSpeak
// ═══════════════════════════════════════════════════════

#include <DHT.h>
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>

// ═══════════════════════════════════════════════════════
// WIFI CONNECTION
// ═══════════════════════════════════════════════════════
```

```
const char* WIFI_SSID = "YOUR_WIFI_NAME";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";

// ═══════════════════════════════════════════════════════
// THINGSPEAK
// ═══════════════════════════════════════════════════════

const char* THINGSPEAK_API_KEY = "ZMREWYM60P7UZ4SC";
const char* THINGSPEAK_SERVER = "http://api.thingspeak.com/update";

// ═══════════════════════════════════════════════════════
// PIN DEFINITIONS
// ═══════════════════════════════════════════════════════

// SW-420 Vibration Sensor
#define VIBRATION_PIN D5

// DHT11
#define DHT_PIN D6
#define DHT_TYPE DHT11

// ACS712
#define CURRENT_PIN A0

// ═══════════════════════════════════════════════════════
// DHT OBJECT
```

```
// ============================================================

DHT dht(DHT_PIN, DHT_TYPE);

// ============================================================
// MACHINE HEALTH THRESHOLDS
// ============================================================

#define TEMP_WARNING 40.0
#define TEMP_CRITICAL 50.0

#define CURRENT_WARNING 2.0
#define CURRENT_CRITICAL 4.0

// ============================================================
// ACS712 SETTINGS
// ============================================================

// ACS712 5A  -> 0.185
// ACS712 20A -> 0.100
// ACS712 30A -> 0.066

#define ACS_SENSITIVITY 0.100

float zeroCurrentVoltage = 2.50;

// ============================================================
```

```
// THINGSPEAK TIMER
// ============================================================

unsigned long lastThingSpeakUpdate = 0;

const unsigned long THINGSPEAK_INTERVAL = 15000;

// ============================================================
// SETUP
// ============================================================

void setup()
{
  Serial.begin(9600);

  // Vibration sensor
  pinMode(VIBRATION_PIN, INPUT);

  // Current sensor
  pinMode(CURRENT_PIN, INPUT);

  // DHT11
  dht.begin();

  // ============================================================
  // WIFI CONNECTION
  // ============================================================
```

```
Serial.println();
Serial.println("═══════════════════════════════════════════");
Serial.println(" PREDICTIVE MAINTENANCE SYSTEM");
Serial.println(" MACHINE HEALTH MONITORING");
Serial.println("═══════════════════════════════════════════");

Serial.println("Connecting to WiFi...");

WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

while (WiFi.status() != WL_CONNECTED)
{
  delay(500);
  Serial.print(".");
}

Serial.println();
Serial.println("WiFi Connected!");

Serial.print("IP Address: ");
Serial.println(WiFi.localIP());

Serial.println("System Initializing...");

delay(3000);
```

```
  Serial.println("System Ready");
  Serial.println("---------------------------------------");
}

// ================================================================
// READ ACS712 CURRENT
// ================================================================

float readCurrent()
{
  const int samples = 100;

  float sum = 0;

  for (int i = 0; i < samples; i++)
  {
    int sensorValue = analogRead(CURRENT_PIN);

    float voltage = sensorValue * (3.3 / 1023.0);

    sum += voltage;

    delay(2);
  }

  float averageVoltage = sum / samples;
```

```
  float current =
    (averageVoltage - zeroCurrentVoltage)
    / ACS_SENSITIVITY;

  if (current < 0)
  {
    current = -current;
  }

  return current;
}

// ============================================================
// LOOP
// ============================================================

void loop()
{
  // ============================================================
  // READ DHT11
  // ============================================================

  float temperature = dht.readTemperature();
  float humidity = dht.readHumidity();

  // ============================================================
  // READ VIBRATION
```

```
// ================================================================

int vibrationState = digitalRead(VIBRATION_PIN);

// ================================================================
// READ CURRENT
// ================================================================

float current = readCurrent();

// ================================================================
// CHECK DHT ERROR
// ================================================================

if (isnan(temperature) || isnan(humidity))
{
  Serial.println("DHT11 Sensor Error!");
}

// ================================================================
// MACHINE HEALTH ANALYSIS
// ================================================================

bool temperatureWarning = false;
bool temperatureCritical = false;

bool currentWarning = false;
```

```
bool currentCritical = false;

bool vibrationWarning = false;

// Temperature
if (!isnan(temperature))
{
  if (temperature >= TEMP_WARNING)
  {
    temperatureWarning = true;
  }

  if (temperature >= TEMP_CRITICAL)
  {
    temperatureCritical = true;
  }
}

// Current
if (current >= CURRENT_WARNING)
{
  currentWarning = true;
}

if (current >= CURRENT_CRITICAL)
{
  currentCritical = true;
```

```
}

// Vibration
if (vibrationState == HIGH)
{
  vibrationWarning = true;
}

// ================================================
// MACHINE STATUS
// ================================================

String machineStatus;

if (temperatureCritical ||
    currentCritical ||
    vibrationWarning)
{
  machineStatus = "MAINTENANCE REQUIRED";
}
else if (temperatureWarning ||
         currentWarning)
{
  machineStatus = "WARNING";
}
else
{
```

```
  machineStatus = "HEALTHY";
}

// ═══════════════════════════════════════════════
// SERIAL MONITOR
// ═══════════════════════════════════════════════

Serial.println();

Serial.println("═══════ MACHINE DATA ═══════");

Serial.print("Temperature: ");

if (isnan(temperature))
{
  Serial.println("ERROR");
}
else
{
  Serial.print(temperature);
  Serial.println(" C");
}

Serial.print("Humidity: ");

if (isnan(humidity))
{
```

```
  Serial.println("ERROR");
}
else
{
  Serial.print(humidity);
  Serial.println(" %");
}

Serial.print("Vibration: ");

if (vibrationState == HIGH)
{
  Serial.println("DETECTED");
}
else
{
  Serial.println("NORMAL");
}

Serial.print("Current: ");
Serial.print(current);
Serial.println(" A");

Serial.print("Machine Status: ");
Serial.println(machineStatus);

Serial.println("=====================================");
```

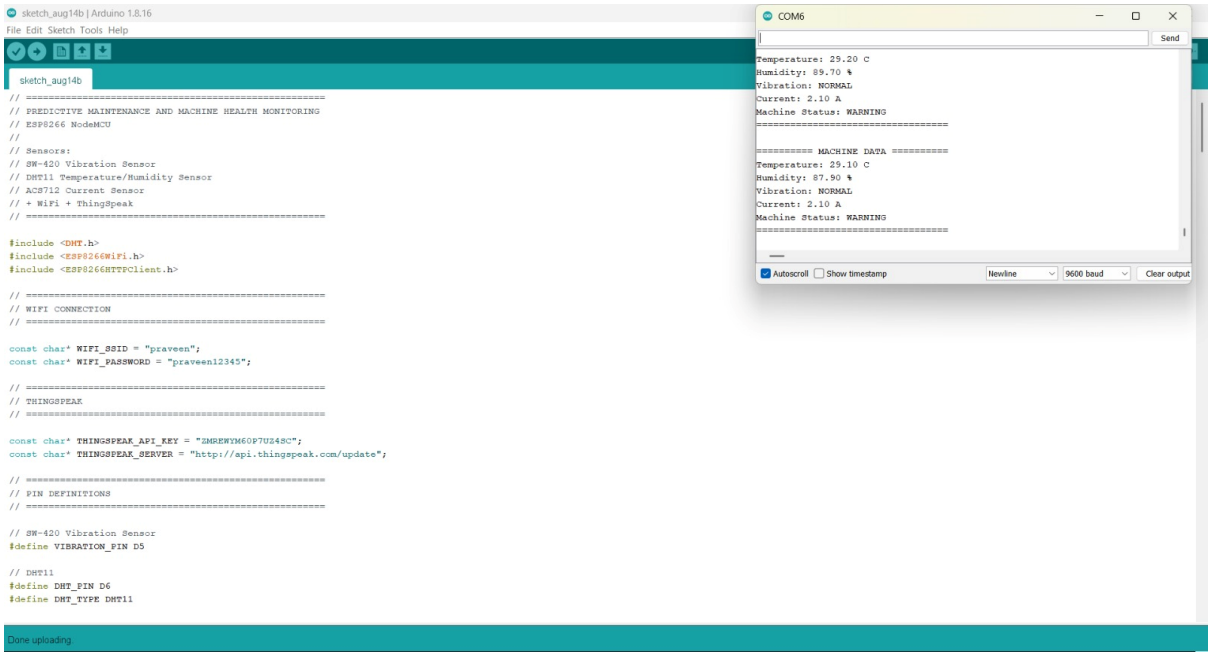
```
// ============================================================
// SEND DATA TO THINGSPEAK
// ============================================================

if (millis() - lastThingSpeakUpdate >= THINGSPEAK_INTERVAL)
{
  lastThingSpeakUpdate = millis();

  if (WiFi.status() == WL_CONNECTED)
  {
    WiFiClient client;
    HTTPClient http;

    String url = String(THINGSPEAK_SERVER);

    url += "?api_key=";
    url += THINGSPEAK_API_KEY;

    // Field 1 = Temperature
    url += "&field1=";
    url += String(temperature);

    // Field 2 = Humidity
    url += "&field2=";
    url += String(humidity);
```

```
// Field 3 = Current
url += "&field3=";
url += String(current);

// Field 4 = Vibration
url += "&field4=";
url += String(vibrationState);

// Field 5 = Machine status
// Numeric values:
// 0 = Healthy
// 1 = Warning
// 2 = Maintenance Required

int statusValue = 0;

if (machineStatus == "WARNING")
{
  statusValue = 1;
}
else if (machineStatus == "MAINTENANCE REQUIRED")
{
  statusValue = 2;
}

url += "&field5=";
url += String(statusValue);
```

```
Serial.println("Sending data to ThingSpeak...");

http.begin(client, url);

int httpCode = http.GET();

if (httpCode > 0)
{
 Serial.print("ThingSpeak Response: ");
 Serial.println(httpCode);

 if (httpCode == 200)
 {
  Serial.println("ThingSpeak Update Successful!");
 }
 else
 {
  Serial.println("ThingSpeak Update Failed!");
 }
}
else
{
 Serial.print("ThingSpeak Error: ");
 Serial.println(http.errorToString(httpCode));
}
```

```
    http.end();
   }
   else
   {
    Serial.println("WiFi Disconnected!");
   }

   Serial.println("-----------------------------------------");
  }

  delay(2000);
}
```

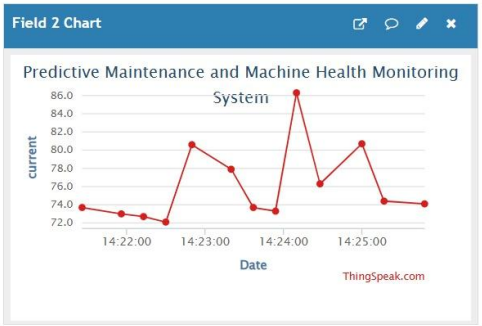
Coding Output:

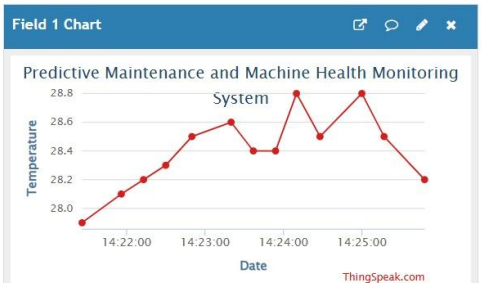


Channel Stats

Created: 5 days ago

Entries: 15





9. System Working:

Power ON the System:

The ESP8266 NodeMCU is powered ON.

Initialize Components:

The PIR sensor, servo motor, buzzer, and LED are initialized.

Monitor Machine Condition:

The temperature sensor continuously monitors the operating temperature.

The vibration sensor continuously monitors machine vibration.

Send Sensor Data:

The sensors send the measured values to the ESP8266 NodeMCU.

Process the Data:

The ESP8266 receives and processes the sensor readings.

Determine Water Level:

As the water level rises, the distance between the sensor and water surface decreases.

Check Machine Health:

The measured values are compared with predefined safety limits.

Abnormal Machine Condition:

If excessive temperature or abnormal vibration is detected:

- Buzzer → ON
- LED → ON
- Relay → ON
- Machine alert → Activated
- Maintenance → Required

Normal Machine Condition:

If the water level is within the safe range

- If the measured values are within the normal range:
    - Buzzer → OFF
    - LED → OFF
    - Relay → OFF
    - Machine condition → Normal

Repeat the Process:

- The sensors continuously monitor the machine, and the ESP8266 continuously checks the machine health.

10. Prototype Implementation:

The sensors are connected to the ESP8266 to monitor the operating condition of a machine. The temperature sensor monitors the machine temperature, while the vibration sensor detects abnormal vibration.

Hardware Setup

1. Connect the Temperature Sensor output → A0 of ESP8266.
2. Connect the Vibration Sensor OUT pin → D4 (GPIO2).
3. Connect the Relay IN pin → D5 (GPIO14).
4. Connect the Buzzer → D6 (GPIO12).
5. Connect the LED → D7 (GPIO13) through a suitable resistor.
6. Connect the sensor VCC pins → suitable power supply.
7. Connect all GND terminals to a common ground.
8. Connect all required components using jumper wires.
9. Upload the program to the ESP8266 using the Arduino IDE.
10. Power ON the prototype.
11. Observe the temperature and vibration sensor readings.
12. Create an abnormal condition by increasing vibration or sensor value.
13. Observe the activation of the buzzer, LED, and relay.
14. The system continuously monitors the machine condition.

Prototype Operation:

The prototype demonstrates automatic machine condition monitoring and early warning. The sensors provide information about temperature and vibration, while the ESP8266 processes the readings and controls the buzzer, LED, and relay according to the programmed condition.

Prototype Working:

- The ESP8266 NodeMCU is powered ON and initializes all connected components.
- The temperature sensor continuously monitors the machine operating temperature.
- The vibration sensor continuously detects machine vibration.
- The sensor readings are sent to the ESP8266.
- The ESP8266 processes and compares the readings with predefined limits.
- If the temperature or vibration level exceeds the selected limit, the system identifies an abnormal machine condition.
- The buzzer turns ON to alert the user.
- The LED provides a visual warning indication.
- The relay connected to D5 (GPIO14) is activated to demonstrate machine safety or shutdown control.
- If the machine parameters are within the normal range, the buzzer and LED remain OFF.
- The system waits for a short interval and reads the sensors again.
- This process continues automatically and continuously, providing real-time machine health monitoring.

11. Bill of Materials:

S:No	Component	Specification	Quantity
1	ESP8266	NodeMCU Wi-Fi Development Board	1
2	Temperature Sensor	Analog Temperature Sensor	1
3	Vibration Sensor	Vibration Detection Module	1
4	Relay Module	1-Channel Relay	1
5	Buzzer	3.3V/5V Buzzer	1
6	LED	Standard Indicator LED	1
7	Resistor	Suitable LED Current Limiting Resistor	1
8	Breadboard	Standard Prototype Board	1
9	Jumper Wires	Male-Male / Male-Female	1 Set
10	USB Cable	Micro-USB	1
11	Power Supply	Suitable Regulated Supply	1

12. Results & Performance Analysis:

- The temperature sensor successfully monitors the machine operating condition.
- The vibration sensor successfully detects changes in machine vibration.
- The ESP8266 NodeMCU successfully receives and processes the sensor readings.
- The system provides continuous monitoring of machine health.
- When abnormal temperature or vibration is detected, the buzzer is activated.
- The LED provides a visual warning indication.
- The relay is activated to demonstrate machine safety or shutdown control.

- When the machine condition returns to the normal range, the warning devices return to their normal state.
- The system provides an automatic and continuous machine health monitoring process.

Results:

- The temperature sensor successfully monitors the machine temperature.
- The vibration sensor successfully detects abnormal machine vibration.
- The ESP8266 NodeMCU successfully receives and processes the sensor data.
- The system continuously monitors the machine condition.
- When an abnormal condition is detected, the buzzer turns ON.
- The LED provides a visual warning indication.
- The relay activates to demonstrate machine protection or shutdown control.
- When the machine condition is normal, the warning devices remain OFF.
- The prototype demonstrates the basic operation of a Predictive Maintenance and Machine Health Monitoring System.

Performance Analysis:

Parameter	Expected Performance
Temperature Monitoring	Continuous
Vibration Monitoring	Continuous
Sensor Response	Real-time Reading
Abnormal Condition Alert	Buzzer ON
Visual Warning	LED ON
Machine Protection	Relay ON
Normal Condition	Buzzer OFF and LED OFF